

Final Report: Score-Based Popularity Prediction in Classical Music

DSA 210 — Spring 2025–2026

1. Motivation

What makes a classical piece endure? Is popularity encoded in the notes themselves, or is it a function of marketing, recording history, and the halo of composer fame?

This question sits at the intersection of computational musicology and data science. If musical qualities that drive popularity are recoverable from the score, computational analysis should be able to distinguish widely loved pieces from obscure ones. This idea follows the lineage of “Hit Song Science” in popular music, where algorithms attempt to predict chart success from audio features, but it remains largely untested in the symbolic classical repertoire.

I set out to answer three research questions:

1. **RQ1:** How much variance in classical music popularity can be explained by score-derived features?
2. **RQ2:** How much of the apparent signal is embedded by the identity of the composer rather than the music itself?
3. **RQ3:** Is the relationship between score features and popularity better modeled as continuous (regression) or discrete (tier classification)?

2. Data Source

2.1 Score Corpus

I sourced MIDI files from the `drengskapur/midi-classical-music` dataset on HuggingFace, which contains approximately 4,500 classical MIDI files. I manually curated the corpus to include only pieces with consistent encoding and verified composer attribution.

To prevent models from simply learning “composer X = popular,” I tried to construct a balanced corpus: each composer was capped at 100 pieces, and pieces were selected to span each composer’s full popularity range when possible. The final corpus contains **961 pieces** across **18 composers** and **5 musical eras** (Baroque, Classical, Romantic, Late Romantic, Impressionist).

Era	Composers	Pieces
Baroque	Bach, Vivaldi, Handel	201
Classical	Mozart, Beethoven, Haydn	291
Romantic	Chopin, Schumann, Brahms, Schubert, Liszt, Tchaikovsky, Dvorak, Grieg, Mendelssohn	404
Late Romantic	Rachmaninov	34
Impressionist	Debussy, Ravel	31

2.2 Popularity Labels

Popularity was measured via the Spotify Web API. For each piece, I searched for recordings by title and composer, collected popularity scores (0–100) for all matching tracks, and computed the **maximum** popularity across all recordings.

I chose the maximum rather than the mean or median because the goal is to measure the effect of the notes on popularity. External variance like a viral TikTok clip, an obscure artist’s upload etc. should fall into the portion of variance the notes *can not* explain. Using a mean or median could distort the signal in hard-to-predict ways: a very popular piece recorded by many obscure artists might see its mean dragged down, while a moderately known piece performed mainly by famous artists might score higher than it should.

2.3 Feature Extraction

I developed a Python pipeline using the `music21` library to extract **65 features** from each MIDI file, organized into two groups:

25 static features capture the overall properties of a piece:

- *Tonal (8)*: pitch range, mean, standard deviation, pitch class entropy, interval mean, stepwise ratio, leap ratio, contour change rate
- *Temporal (6)*: total notes, duration, note density, unique durations, duration standard deviation, repetition ratio
- *Harmonic/Structural (11)*: chord change rate and variability, average chord size, vertical range, chord pitch class entropy, tonic-dominant ratio, cadence rate, key change rate, modulation mean, sectional repetition ratio, sectional similarity mean

40 temporal evolution features capture how musical properties change over the course of a piece. Each piece was divided into four quartiles. Eight base features (note count, pitch mean, pitch standard deviation, pitch range, note density, interval mean, stepwise ratio, unique pitches) were computed per quartile (32 values), plus the linear slope across quartiles (8 values). These capture trajectories like “increasing density” or “narrowing pitch range,” properties that static averages miss entirely.

3. Data Analysis

3.1 Exploratory Data Analysis

The popularity distribution is approximately symmetric (mean 54.04, median 54, range 5–88) with slight left skew. A Shapiro-Wilk test confirmed non-normality ($W = 0.976$, $p < 10^{-11}$), justifying non-parametric methods throughout.

I found that 83% of tracks share a popularity score with at least two other tracks by the same composer, creating visible horizontal “bands” in the data. A Kruskal-Wallis test confirmed that composer identity explains approximately 38.3% of rank-variance in popularity ($H = 377.2$, $p < 10^{-70}$), while era has minimal direct effect ($\eta^2 = 0.015$).

Spearman correlations with Bonferroni correction revealed that 34 of 65 features are significantly correlated with popularity, but all individual correlations are modest ($|\rho| \leq 0.31$). Mann-Whitney U tests confirmed that 12 of 25 static features differ significantly between top-quartile and non-top-quartile pieces, with moderate effect sizes. Importantly, Fisher z-tests showed that feature-popularity relationships vary across eras: pitch standard deviation, for example, has a

strong negative correlation with popularity in the Baroque era ($\rho = -0.518$) but only a weak one in the Romantic era ($\rho = -0.134$).

3.2 Regression: Three Approaches

Approach 1: Popularity Similarity Index (PSI). I defined a custom baseline: the cosine similarity between a piece’s standardized feature vector and the centroid of the top 25% most popular pieces. With 14 baseline features (8 tonal + 6 temporal), PSI achieved a Spearman correlation of $r = 0.28$, explaining $\sim 7.8\%$ of popularity variance. Adding 11 harmonic features to reach 25 total yielded only marginal improvement ($r = 0.29$, $\sim 8.5\%$).

Approach 2: Linear Regression. OLS and Ridge regression on 25 static features achieved $\sim 7.4\%$ R^2 in 5-fold cross-validation, converging to the same ceiling as PSI. Two independent methods hitting the same ceiling suggested it might be a genuine limit of linear approaches.

Approach 3: Non-Linear Tree Models. Random Forest and Gradient Boosting broke through the linear ceiling: RF achieved 17.2% and GBR 11.0% on static features alone, and adding temporal features pushed these to 17.9% and 12.9%.

3.3 Disentangling Composer Identity

The improvement from tree models raised the question: were they learning musical patterns or identifying composers? A Random Forest classifier trained on 25 static features to predict composer identity achieved 54.1% accuracy (chance: $\sim 5.5\%$), confirming that the features encode substantial stylistic information.

To test this, I **residualized** the features by removing the per-composer mean from each piece, preserving only within-composer variation:

$$\vec{x}'_i = \vec{x}_i - \vec{x}_c$$

The results were revealing:

Linear models collapsed: OLS/Ridge R^2 dropped from $\sim 7\text{--}8\%$ to approximately 0%. Their entire signal was composer-identity leakage.

Tree models improved: GBR R^2 *increased* from $\sim 11\%$ to $\sim 25\%$ after residualization. Composer identity had been consuming model capacity on a shortcut. Once blocked, tree models found genuine within-composer patterns.

Adding temporal features to residualized static features pushed GBR from 19.3% to 23.1%, confirming that the dynamic arc of a piece carries approximately 4 percentage points of additional signal. The result was robust across 5 random seeds (GBR residualized: $23.7\% \pm 1.4\%$).

In a direct comparison, composer identity alone (OLS on one-hot encoded names) explains 35.0% of variance, score features alone (GBR, residualized) explain 24.6%, and the combined model reaches $\sim 39.0\%$. The gap is substantially smaller than linear analysis suggested ($1.4\times$ vs $4.7\times$).

3.4 Classification: Where the Signal Lives

The $\sim 24\%$ regression ceiling suggested that score features carry real but limited information about exact popularity values. However, the difference between popularity 43 and 48 may be noise, while the difference between “popular tier” and “not popular tier” may be meaningful. I reframed the problem as binary classification: can the model identify pieces in the top popularity quartile?

Results using residualized features and stratified 5-fold cross-validation:

Task	Model	AUC
Top 25% vs Rest	RF	0.84
Top 25% vs Bottom 25%	GBR	0.89
Top 10% vs Rest	RF	0.80

The top-25% vs rest result (AUC 0.84) is the primary metric. The extremes comparison (AUC 0.89) provides supporting evidence that the signal strengthens when ambiguous cases are removed.

3.5 Verification

Three checks confirmed the classification signal is genuine:

Leakage check: Raw AUC (0.838) $\approx$ Residualized AUC (0.842). The classification ability is not a composer-identity artifact.

Precision/Recall: 86% precision (when the model predicts popular, it is correct 86% of the time) and 45% recall (roughly half of popular pieces are identified). This asymmetry has a natural interpretation: approximately half of popular pieces are popular for reasons visible in the score, and the other half for reasons the score cannot capture.

Calibration: The model is consistently underconfident. When it assigns a 75% probability of being in the top quartile, the actual frequency is 97%. Pieces that score highly on the model’s internal ranking are almost certainly popular.

4. Findings

4.1 The Notes Matter But Differently Than Expected

Score-based features explain approximately 24% of continuous popularity variance when modeled non-linearly and controlled for composer identity. Linear models struggle at $\sim 8\%$, and the figure drops to essentially 0% when accounting for composer-identity leakage. When the question is reframed from “how popular is this piece?” to “is this piece in the popular tier?”, score features achieve an AUC of 0.84 with 86% precision.

4.2 Composer Identity Leakage Is Pervasive

The most methodologically important finding is that apparent linear correlations between score features and popularity are almost entirely mediated by composer identity. A feature like “pitch range” correlates with popularity partly because different composers use different pitch ranges and have different fame levels. Residualization is essential for any honest assessment of score-level predictive power.

4.3 Classification Outperforms Regression

The disproportionate classification success (AUC 0.84 vs R^2 0.24) reveals the structure of the relationship: score features define a *region* in feature space where popularity is possible, not a gradient along which popularity increases. External factors like performance quality, cultural associations, algorithmic amplification etc. determine which pieces within that region actually become popular.

4.4 What Popular Pieces Sound Like

From the data, popular classical pieces tend to be shorter, harmonically consistent but chromatically rich, and tonally focused. The five strongest static correlates are:

Feature	Correlation	Direction
chord_change_std	−0.256	Consistent harmonic rhythm → popular
duration	−0.254	Shorter → popular
chord_pc_entropy	+0.250	Rich chord vocabulary → popular
pitch_std	−0.231	Narrower pitch focus → popular
pitch_range	−0.223	Narrower range → popular

Popular pieces also show steeper dynamic arcs, particularly in note density, suggesting that “building toward a climax” is a measurable predictor of popularity.

4.5 “Popular” Means Different Things Across Eras

Feature-popularity relationships reverse between eras. In the Baroque era, narrower pitch range correlates with popularity; in the Impressionist era, wider range does. This means any pooled prediction model is averaging over fundamentally different aesthetic standards, and era-specific models may perform better.

5. Limitations and Future Work

5.1 Limitations

MIDI encoding quality. Public MIDI datasets vary in encoding fidelity. While the manual curation aimed for consistency, differences in ornament realization and voice-leading detail may introduce noise.

Symbolic-only analysis. Our features capture notation but not orchestration, timbre, or dynamics. Pieces that derive their appeal from orchestral color (as with much of Tchaikovsky’s output) are disadvantaged in the framework.

Sample size imbalance. Several composers are represented by fewer than 20 pieces, and the Impressionist era has only 31 pieces. Era-level findings should be interpreted with caution.

Popularity metric. Spotify popularity scores are rolling metrics influenced by recent algorithmic placement. Findings describe what predicts current streaming popularity, not historical significance or musicological importance.

Temporal granularity. Dividing pieces into four quartiles is a coarse approximation. Finer segmentation or attention-based approaches might capture more nuanced temporal structure.

5.2 Future Work

Audio features. Incorporating audio-level features (timbre, dynamics, recording quality) could capture the performance dimension that score analysis misses, potentially explaining some of the 45% of popular pieces my model fails to recall.

Corpus expansion. Expanding beyond 18 composers and 961 pieces, particularly in underrepresented eras (Impressionist, Late Romantic, 20th century), would strengthen era-level conclusions.

Generalization testing. Testing whether the classification boundary generalizes to newly composed pieces would be the strongest validation of the “notes matter” hypothesis.

Deeper temporal modeling. Replacing quartile-based features with sequence models (RNNs, Transformers) that operate directly on note sequences could capture temporal patterns the slope features miss.

Cross-platform validation. Comparing Spotify popularity with YouTube view counts, concert programming frequency, or historical performance records would test whether the findings are Spotify-specific or reflect broader cultural patterns.

References

1. Cuthbert, M. S., & Ariza, C. (2010). music21: A toolkit for computer-aided musicology and symbolic music data. *Proceedings of the 11th International Society for Music Information Retrieval Conference (ISMIR)*.
2. Spotify Web API. <https://developer.spotify.com/documentation/web-api>